Common Core: Phase 2 Verification

Lab V3: Constrained-Random Verification of the Cache

Classes, Constraints, Functional Coverage and Testplans

ASIC Design, Verification and Implementation Onboarding

Kerim Turak

October 18, 2026

Lab objective: Verify the Lab R4 data cache with class-based constrained-random stimulus, three independent checks (read data, final memory and hit/miss), design assertions and a functional coverage plan, close coverage over a regression, and compare the measured performance with your Lab R4 predictions.

Duration	2 weeks (approximately 35–40 hours)
Prerequisites	Labs R4 and V2
Designs	`dcache` (verified; RTL changed only to fix bugs), `arb_mem` and `mem_if` (reused unchanged)
Reused in	P3 (PPA of `dcache`), T-MOD and T-DV1 (transactions, tests and coverage plan), T-DV2, T-AM2 (latency statistics)

Contents

1 Overview **3**

2 The Design Under Test and the Environment **4**

2.1 System Structure . . . 4

2.2 Testbench Structure . . . 4

3 Classes and Constrained Randomization **5**

3.1 A Transaction Class . . . 6

3.2 Inheritance and Inline Constraints . . . 6

3.3 Rules . . . 7

4 Verification Strategy **8**

4.1 Transparency: What the Reference Model Needs . . . 8

4.2 Three Independent Checks . . . 8

4.3 The Tag Model . . . 8

4.4 Latency Statistics . . . 9

4.5 Memory Preload: Frontdoor and Backdoor . . . 9

5 Tests and Plusargs **10**

6 Functional Coverage **11**
6.1 Covergroup Basics . . . 11
6.2 Required Coverage Plan . . . 11
6.3 The Coverage Class and the Tool-Note Fallback . . . 12

7 Design Assertions **13**
7.1 Binding from `dv/sva/` . . . 13
7.2 Assertion Fires First . . . 14

8 Coverage Closure and the Testplan **14**
8.1 Coverage Closure as the Completion Criterion . . . 14
8.2 The Testplan with Coverage Items . . . 14
8.3 Regression . . . 15

9 Measured Performance versus Lab R4 Predictions **15**

10 Stretch Goal: Verifying the 2-Way Cache **16**

11 Requirements / Tasks **16**

12 Repository and Flow **17**

13 Deliverables **18**

14 Assessment **18**

15 Review Questions **18**

16 References **19**

1 Overview

A cache is the classic example of a block whose bugs are easy to write and hard to see. A wrong dirty bit, a write-back to the wrong address or a line that is never refilled can stay invisible for millions of cycles, because the cache keeps returning the right value from its own copy until the line is evicted much later. In a real chip such bugs surface as data corruption in software, long after tape-out. Verification of a cache therefore needs more than "read back what you wrote": it needs checks that look at memory after the fact, a prediction of *what the cache should have done*, and evidence that the interesting situations actually occurred.

In Lab R4 you designed `dcache` and checked it with a small directed smoke testbench and hand-predicted hit rates. In this lab you verify it properly. You will:

- describe stimulus with **classes and constraints** instead of `$urandom_range` calls spread over tasks;
- build a scoreboard with **three independent checks** and a small **tag model** that predicts every hit and miss;
- preload the memory through a **frontdoor** port (and optionally a backdoor);
- write a **functional coverage** model and use **coverage closure**, not a transaction count, as the completion criterion;
- bind **design assertions** for the cache controller from `dv/sva/`;
- record everything in a **testplan** that maps features to tests, assertions and coverage items;
- **measure** the cache's performance with the RTL counters and compare it with your Lab R4 predictions.

Where this lab sits. It builds on Lab R4 (the design, its performance counters and the hand predictions), Lab V1 (structure of `dv/`, plusargs, `regress`, SVA) and Lab V2 (virtual interfaces, testbench arrays, `fork/join`, scoreboards in acceptance order, `mem_if` protocol assertions). Its outputs are reused later: the verified `dcache` is synthesized in Lab P3; the transaction classes, tests and coverage plan are the starting point of the class-based environment in Lab T-DV1 (which replaces the three checks with the cache model from Lab T-MOD) and of the UVM environment in Lab T-DV2; the latency statistics calibrate the cycle-approximate model in Lab T-AM2.

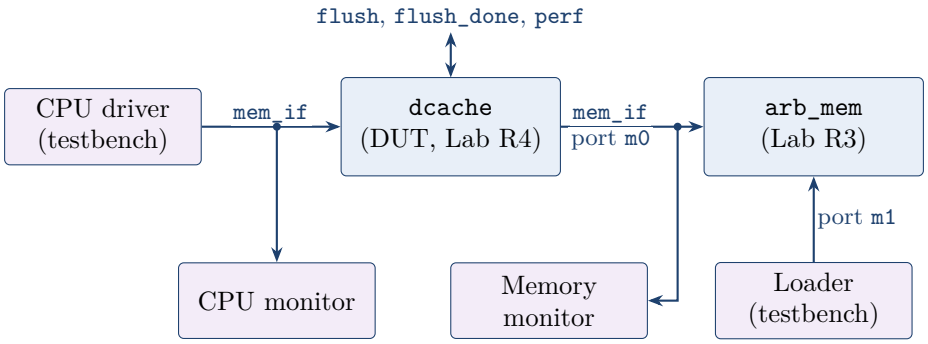
By the end of this lab you should be able to:

1. Write transaction classes with `rand` fields, constraints, knobs, inheritance and inline constraints, and avoid the handle-sharing pitfall.
2. Explain why a cache reference model can be a flat memory, and what each of the three checks can and cannot find.
3. Predict hits and misses with a tag model and compare the prediction with what the DUT did on its memory port.
4. Write a coverage model with coverpoints, bins and crosses, and decide from it when verification is complete.
5. Write design assertions on internal state and attach them with `bind` without touching the RTL.
6. Write a testplan whose every line points to a test, an assertion or a coverage item.
7. Measure hit rate, miss penalty and average latency, and explain the differences from a hand prediction.

2 The Design Under Test and the Environment

2.1 System Structure

The DUT is the Lab R4 `dcache`, unchanged, with `arb_mem` from Lab R3 as main memory. The testbench file list includes the `arb_mem` file list with a nested `-f`; nothing is copied.



- The CPU side of the cache is a `mem_if` slave; the memory side is a `mem_if` master connected to port `m0` of `arb_mem`. The `mem_if` protocol assertions written in Lab V2 are part of the interface, so they check both cache ports automatically.
- Port `m1` of `arb_mem` belongs to the testbench *loader*. It preloads the memory before the test and reads it back after the final flush. It must stay idle while the cache is in use: a second master changing memory behind the cache's back would make the cached copy stale. That problem, *coherence*, is out of scope (Lab R4).
- Two monitors observe the two `mem_if` instances. They are the only source of information for the checks: the scoreboard never peeks into the DUT.

Recall from Lab R4 the key behavior that shapes the whole testbench: **the cache accepts a CPU request only when it hits**. On a miss, `cpu.req_ready` stays low while the cache evicts (if dirty) and refills the line; then the held request hits and is accepted. All memory traffic caused by a request therefore happens *between the arrival of the request and its acceptance*.

2.2 Testbench Structure

The testbench follows the Lab V1/V2 structure. What changes in this lab is that transactions and the coverage collector become *classes*. The driver and monitors stay tasks with virtual interfaces, as in Lab V2; a fully class-based, layered environment is the topic of Lab T-DV1. Figure 1 shows how the parts connect; the table below lists where each one lives.

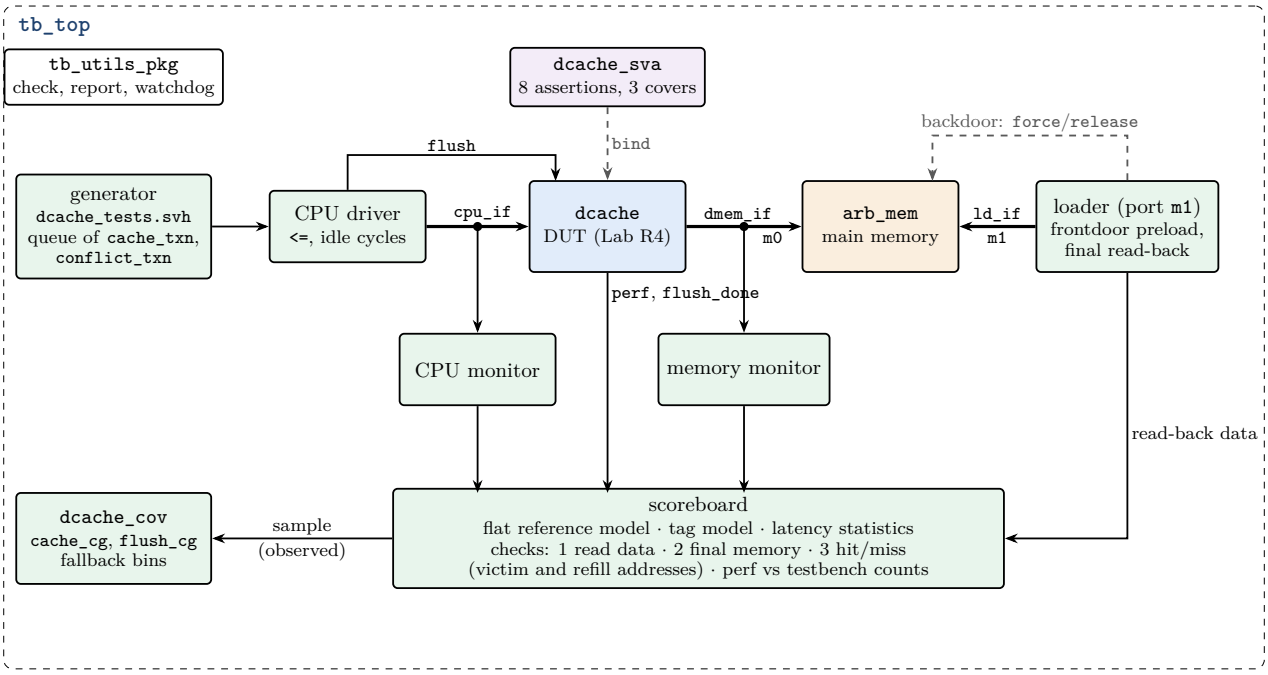


Figure 1: Lab V3 testbench architecture. Green: testbench (classes for the transactions and the coverage model, tasks for driver, monitors, loader and scoreboard); blue: the DUT; orange: the memory reused from Lab R3; pink: the design-assertion checker attached with `bind`. The scoreboard sees only what the two monitors, the loader and the `perf` port report; it never peeks into the DUT.

Component	**Implemented as**	**Responsibility**
Transactions	classes `cache_txn`, `conflict_txn`, ... (`dv/env/dcache_txn.svh`)	Random fields, knobs and constraints
Sequence generator	task per test (`dv/tests/`)	Fill a queue of transaction objects according to `+TEST=`
CPU driver	task, `virtual mem_if`	Drive each transaction with `<=`, honor idle cycles and the handshake
Loader	tasks on port `m1`	Preload memory; read it back after the final flush
CPU monitor	task	Accepted requests, read responses, request latency
Memory monitor	task	Accepted memory-side requests between request arrival and acceptance
Reference model	associative array	Flat memory, updated by accepted CPU writes
Tag model	dynamic arrays	Predicts `HIT`, `MISS_CLEAN`, `MISS_DIRTY`
Coverage	class `dcache_cov` (`dv/cov/`)	Covergroup or fallback counters; coverage report
Assertions	`dcache_sva` bound from `dv/sva/`	Controller rules (Section 7)
Watchdog, report	`tb_utils_pkg` (Lab V1)	Timeout, error counters, `TEST PASSED`/`TEST FAILED`

3 Classes and Constrained Randomization

Until now, stimulus came from `$urandom_range` calls inside tasks. That works for a few fields, but it becomes hard to express rules such as "70% of the addresses fall into one index" or "a write is

never followed by a read of a different line". Constrained randomization lets you describe the *legal and interesting* space and leaves the solving to the simulator.

Test term — Constrained-random test. Random stimulus whose legal ranges and distributions are described with class constraints (`rand`, `constraint`, `dist`, `inside`), so that the simulator's solver spends the run on interesting cases (one hot index, a chosen share of writes) instead of uniformly random noise. A pass means nothing was wrong in the cases that were generated — which is why it is paired with functional coverage, which tells you which cases were.

3.1 A Transaction Class

Declare the classes in a `.svh` file included inside `tb_top`, so they can use the module parameters (`ADDR_W`, `IDX_W`, ...) directly. Parameterized classes in packages, which remove this restriction, are introduced in Lab T-MOD.

```
// hw/ip/dcache/dv/env/dcache_txn.svh  (included in tb_top)
class cache_txn;
  rand bit                 we;
  rand logic [ADDR_W-1:0] addr;
  rand logic [DW-1:0]     wdata;
  rand int unsigned       idle;

  // knobs, set by the test before randomize()
  int unsigned       wr_pct  = 50;
  logic [ADDR_W-1:0] addr_lo = '0;
  logic [ADDR_W-1:0] addr_hi = '1;

  constraint c_we   { we dist { 1 := wr_pct, 0 := 100 - wr_pct }; }
  constraint c_addr { addr inside {[addr_lo : addr_hi]}; }
  constraint c_idle { idle dist { 0 := 70, [1:3] :/ 30 }; }

  function string to_string();
    return $sformatf("%s addr=%h wdata=%h idle=%0d",
                     we ? "WR" : "RD", addr, wdata, idle);
  endfunction
endclass
```

- `rand` fields are chosen by `randomize()`; plain fields (the *knobs*) are inputs to the constraints and are set by the test, typically from plusargs.
- `dist` gives weights: `:=` assigns the weight to each value of a range, `:/` divides it over the range. `c_idle` uses `:/`, so 30 % of the requests have idle cycles (with `:=` the range would get 3×30).
- `inside` restricts a field to a set or a range.

3.2 Inheritance and Inline Constraints

```
// All addresses map to one index: forces conflict misses and evictions
class conflict_txn extends cache_txn;
  logic [IDX_W-1:0] hot_index;
  constraint c_conflict { addr[OFF_W +: IDX_W] == hot_index; }
endclass

// Inline constraint for one call only
if (!t.randomize() with { we == 0; addr[OFF_W-1:0] == 0; })
  $fatal(1, "randomize failed");
```

```
// Switch a constraint off for a directed corner case
t.c_idle.constraint_mode(0);
```

A derived class *adds* constraints to those of its base class; all of them must hold at the same time. A test that needs a different stimulus creates an object of a derived class and keeps the rest of the testbench unchanged. This idea grows into the blueprint pattern of Lab T-DV1 and the factory of Lab T-DV2.

3.3 Rules

- **Always check the result of `randomize()`.** It returns 0 when the constraints cannot be solved and leaves the fields unchanged; ignoring it silently repeats the previous values.
- **Create a new object for every transaction.** A class variable is a handle (a reference), not the object itself:

```
// BAD: the queue holds n handles to the SAME object,
// so every element shows the last randomized values
cache_txn t = new();
repeat (n) begin
  void'(t.randomize());
  seq.push_back(t);
end

// GOOD: a new object per transaction
repeat (n) begin
  cache_txn t = new();
  if (!t.randomize()) $fatal(1, "randomize failed");
  seq.push_back(t);
end
```

- **Randomization must be reproducible.** The simulation seed (`SEED`, Lab V1) determines every `randomize()` result. Print the seed at the start; a failure must reproduce with the same seed and the same plusargs.
- **Keep knobs out of constraints' way.** Check knob values from plusargs before use (`addr_lo <= addr_hi`, `wr_pct <= 100`); an impossible knob combination makes every `randomize()` fail.

Tool note. Constrained randomization in Verilator requires a recent version and an external constraint solver. The coordinator confirms support before the lab starts. If it is not available, implement the same knobs in a `function void my_randomize()` method using `$urandom_range`, and keep the class structure unchanged, so that switching to `randomize()` later touches only that method.

On the Synopsys cloud: VCS (`make sim SIM=vcs`, the cloud default) fully supports `randomize()`, `dist`, `inside`, inline `with` constraints, `constraint_mode()` and inheritance of constraints; use it for this lab. Verilator 5.013 on the cloud has no constraint solver: it stops with `Unsupported: dist` and `Unsupported: randomize() 'with' constraint`. The seed of `make sim SEED=`n reaches VCS as `+ntb_random_seed=`n, so every `randomize()` result is reproducible.

4 Verification Strategy

4.1 Transparency: What the Reference Model Needs

Software must not be able to tell whether a cache exists. Every value read through the cache must equal the value a plain memory would return. Therefore the reference model does *not* model the cache at all: it is the flat memory model from Lab V2 (an associative array), updated by every write *accepted* by the cache, in acceptance order. This is the most important design decision of the scoreboard; explain it in `doc/verification.md`.

4.2 Three Independent Checks

1. **Read data check.** Every CPU read response is compared against the flat reference model.
2. **Final memory check.** After the test, the testbench pulses `flush`, waits for `flush_done`, reads the whole memory through the loader port and compares every word with the reference model. Only this check finds a write that was never written back — the read check cannot, because the cache still returns the correct value from its own copy.
3. **Hit/miss check.** A cache that misses on every access passes checks 1 and 2: it is functionally correct, but useless. The testbench therefore contains a small *tag model* that predicts the outcome of each access: `HIT`, `MISS_CLEAN` or `MISS_DIRTY`. The memory monitor classifies what the DUT actually did from the memory-side traffic between the arrival of the request and its acceptance: no traffic is a hit, reads only are a clean miss, writes followed by reads are a dirty miss. Prediction and observation must match for every access.

Check	Finds	Cannot find
Read data	Wrong refill data, wrong word selected, lost write hit	A dirty line that is never written back; useless misses
Final memory	Missing or wrong write-backs, flush bugs	Which access caused the error; useless misses
Hit/miss	Missing hits, unnecessary misses, wrong dirty state, unnecessary write-backs	Wrong data values

4.3 The Tag Model

The tag model holds only what decides hit or miss: valid, dirty and tag per line. It is updated in DUT acceptance order, like the reference model.

```
typedef enum { HIT, MISS_CLEAN, MISS_DIRTY } result_e;

bit               tm_valid [];   // sized new[NUM_LINES] at reset
bit               tm_dirty [];
logic [TAG_W-1:0] tm_tag   [];

function automatic result_e tm_access(bit we, logic [ADDR_W-1:0] addr);
  int unsigned      idx = addr[OFF_W +: IDX_W];
  logic [TAG_W-1:0] tag = addr[ADDR_W-1 -: TAG_W];
  result_e          res;
  if (tm_valid[idx] && tm_tag[idx] == tag) res = HIT;
  else if (tm_valid[idx] && tm_dirty[idx])  res = MISS_DIRTY;
  else                                      res = MISS_CLEAN;
  if (res != HIT) begin                     // refill
    tm_valid[idx] = 1'b1;  tm_tag[idx] = tag;  tm_dirty[idx] = 1'b0;
  end
```

```
  if (we) tm_dirty[idx] = 1'b1;              // write-allocate, write-back
  return res;
endfunction
```

The observed classification goes one step further than "reads or writes": for a dirty miss, the write-back addresses must be the `LINE_WORDS` words of the *victim* line (old tag), and the refill addresses the words of the *requested* line. Record the victim tag in the tag model before it is overwritten and check both address sets.

4.4 Latency Statistics

The CPU monitor measures, for every request, the number of cycles from the first cycle of `req_valid` to acceptance, and stores it per result class in dynamic arrays or queues (Lab V2). At the end it reports minimum, average and maximum latency for `HIT`, `MISS_CLEAN` and `MISS_DIRTY`. These numbers are the measured miss penalties used in Section 9 and, later, to calibrate the timing model of Lab T-AM2.

4.5 Memory Preload: Frontdoor and Backdoor

Before the first CPU request, the loader writes a known pattern (for example, each word equal to its address XOR a constant) into the whole memory through port `m1`, and the reference model is initialized with the same pattern. Every address is then defined, so every read can be checked — unlike Lab V2, no read is skipped as "never written".

Loading through a real port is called *frontdoor* access. Writing the memory array directly by hierarchical reference (`u_mem.mem[i] = ...`) is *backdoor* access: much faster, but it bypasses the port logic and depends on the internal name of the array.

	Frontdoor	**Backdoor**
Mechanism	Real transactions on port `m1`	Hierarchical assignment to the storage array
Cost	`2**ADDR_W` cycles at least	Zero simulation time
Exercises the design	Yes (`arb_mem` write path)	No
Breaks when	Port protocol changes	Internal array is renamed, restructured, or replaced by an SRAM macro

A practical detail: `arb_mem` writes `mem` in an `always_ff` block, which must be the only writer of that variable. VCS therefore rejects a hierarchical assignment, `$readmemh` or `$deposit` into it (`Error-[ICPD] Illegal combination of drivers`). `force`/`release` is allowed: a `generate` loop in `tb_top` with one `force u_mem.mem[a] = ...; release u_mem.mem[a];` per word, started by an event, loads the array in zero time; after the release each word keeps its value until the next write through the port.

Implement the frontdoor loader and select it by default. Optionally add a backdoor version (`+PRELOAD=backdoor`) and compare the run times of a short test. The final memory check always reads through the frontdoor.

5 Tests and Plusargs

Test term — Smoke test. A short directed test of the most basic behavior, compared with known results. Here `smoke` replays the Lab R4 hit / clean miss / dirty miss sequence through the new environment: if it fails, the environment or the DUT is broken and nothing else is worth running. It is the default test and the first line of the regression.

Test term — Random test. Many transactions drawn from a seeded random generator, checked by the reference model and the scoreboard. It finds bugs nobody thought of; the printed seed reproduces a failure exactly. `fits`, `conflict` and `random` are random tests whose constraints aim them at different parts of the cache.

`+TEST=`	Scenario
`smoke`	A few directed reads and writes: hit, clean miss, dirty miss (the Lab R4 cases, now checked by all three checks)
`sequential`	Reads with stride 1 over four times the cache capacity
`fits`	Random reads and writes inside a working set that fits in the cache
`conflict`	`conflict_txn`: all addresses on one index, several tags
`random`	Random reads and writes over the whole address space
`flush`	Flush with no dirty line, with some dirty lines and with all lines dirty

What each test is meant to show:

- `smoke` — the environment works: all three checks see the R4 cases and agree with the DUT.
- `sequential` — spatial locality: one clean miss per line, then hits on the same line (coverage item `C4`); the hit rate you predicted in Lab R4.
- `fits` — only cold misses: after the first touch of each line everything hits, and write hits never touch memory.
- `conflict` — conflict misses and dirty evictions back to back on one index: the eviction path under stress.
- `random` — everything mixed over the whole memory, long runs and periodic flushes: bugs nobody thought of.
- `flush` — a corner-case test: the flush with 0, 1, some and all lines dirty, and the data after each flush.

Plusarg	Default	Meaning
`+NUM_TXN=`	2000	CPU transactions per test
`+WR_PCT=`	50	Percentage of writes (knob `wr_pct`)
`+IDLE_PCT=`	30	Percentage of transactions with idle cycles before them
`+ADDR_RANGE=`	`2**ADDR_W`	Addresses used are `0 .. ADDR_RANGE-1`
`+FLUSH_EVERY=`	0 (off)	Issue a flush every n transactions during `random`
`+PRELOAD=`	`frontdoor`	`frontdoor` or `backdoor`

All plusargs are read with the `tb_utils_pkg` helpers and printed with the configuration at the start of the simulation, as required since Lab V1. Plusargs set knobs; they never replace constraints. For example, `fits` sets `addr_hi` so that the working set is at most the cache capacity, *computed from the parameters*, not hard-coded.

6 Functional Coverage

Test term — Coverage closure. Not a test but the completion criterion of this lab: the coverage plan lists the situations that must have happened (and been checked), and the regression is extended — knobs, constraints, directed sequences — until every planned bin is hit in every configuration. A closed plan proves the tests reached the planned cases; it says nothing about cases nobody planned.

Assertions tell you that nothing illegal happened. Coverage tells you whether the important things happened at all. A regression is complete when every planned coverage bin has been hit and every check passes — not when a certain number of transactions has run.

6.1 Covergroup Basics

```
covergroup cache_cg with function sample(bit we, result_e res,
                                         logic [IDX_W-1:0] idx);
  cp_op  : coverpoint we  { bins rd = {0}; bins wr = {1}; }
  cp_res : coverpoint res;                 // one bin per enum value
  cp_idx : coverpoint idx;                 // one bin per index value
  x_op_res : cross cp_op, cp_res;          // 2 x 3 = 6 bins
endgroup

cache_cg cg = new();
// in the monitor, once per accepted CPU request:
cg.sample(we, observed_result, idx);
```

- A **coverpoint** watches one value; its **bins** are the values or ranges you want to see. Without explicit bins, each value of an enum or a small vector gets its own bin.
- A **cross** counts combinations of coverpoints. Crosses are where real bugs hide: "write that causes a dirty miss" is a cross bin, not a single value.
- `ignore_bins` removes values that do not matter; `illegal_bins` turns values that must never occur into errors (prefer an assertion for those).
- Sampling explicitly with `sample()` at the moment a transaction completes is clearer than sampling on every clock. Sample the *observed* result, after the hit/miss check has confirmed it: coverage of something that was not checked is worthless.

6.2 Required Coverage Plan

ID	Item	Bins
C1	Operation × result	All 6 combinations of read/write and hit/clean miss/dirty miss
C2	Index	Every index accessed at least once
C3	Offset on miss	Miss on every word offset of a line (the requested word is not always word 0)
C4	Back-to-back	Hit directly after a miss on the same line; miss directly after a miss
C5	Flush	Flush with 0 dirty lines, 1 dirty line and all lines dirty
C6	Request timing	Request with 0 idle cycles after the previous acceptance; request after idle cycles

The IDs are referenced from the testplan (Section 8).

6.3 The Coverage Class and the Tool-Note Fallback

Wrap the coverage model in a class, `dcache_cov`, in `dv/cov/dcache_cov.svh`. The rest of the testbench only calls its methods, so the implementation behind them can change. The covergroup keeps the name `cache_cg` from the basics above; Lab T-DV1 reuses this class unchanged and Lab T-DV2 refers to `cache_cg` by name.

```
class dcache_cov;
`ifdef DV_COVERGROUP
  covergroup cache_cg with function sample(bit we, result_e res,
                                           int unsigned idx, int unsigned off);
    // C1..C4, C6 as in the plan (cp_op, cp_res, cp_idx, x_op_res, ...)
  endgroup
`endif
  int unsigned bin_cnt [string];      // fallback: one counter per bin name
                                      // ('bins' is a keyword)

  function new();
`ifdef DV_COVERGROUP
    cache_cg = new();
`endif
    // pre-create every planned bin with count 0, so unhit bins are reported
  endfunction

  function void sample_access(bit we, result_e res,
                              int unsigned idx, int unsigned off);
`ifdef DV_COVERGROUP
    cache_cg.sample(we, res, idx, off);
`endif
    bin_cnt[$sformatf("C1_%s_%s", we ? "wr" : "rd", res.name())]++;
    bin_cnt[$sformatf("C2_idx_%0d", idx)]++;
    if (res != HIT) bin_cnt[$sformatf("C3_off_%0d", off)]++;
  endfunction

  function void sample_flush(int unsigned n_dirty);  // C5
  function void report(string file);                  // print and write
endclass
```

Tool note. Verilator's support for `covergroup` is limited. If the installed version does not collect covergroup coverage, the counters in `bin_cnt` *are* the coverage model: `report()` prints every bin with its count, marks bins with count 0, and writes the same list to `cov_<test>_s<seed>.txt` next to the simulation log. Keep the covergroup in the source, guarded by `DV_COVERGROUP`, so the testbench also runs on simulators that support it (`make sim DEFINES=DV_COVERGROUP`).

On the Synopsys cloud: VCS collects covergroups completely, together with code coverage. Compile the covergroup with `DEFINES=DV_COVERGROUP` (or define the macro inside `` `ifdef VCS `` in `dcache_cov.svh`, so `SIM=vcs` always has it). `make sim COV=1` adds `-cm line+cond+fsm+tgl+branch+assert` (`COV_METRICS`) and writes every run into `build/dcache/<cfg>/cov/simv.vdb`; `make cov` runs URG on it: `urg_report/dashboard.txt` gives the line, condition, toggle, FSM, branch and assertion scores (code coverage) and `groups.txt` the covergroup bins (functional coverage). For an interactive view of the holes use Verdi: `verdi -cov -covdir build/dcache/<cfg>/cov/simv.vdb &`. Run `make regress COV=1`, then `make cov PARAMS=...` once per configuration: the URG merge replaces `cov_merge.py`, which stays the method for simulators without covergroups. Verilator 5.013 cannot compile this environment (see the randomization box), so its line coverage is not available here.

Pre-creating all bins with count 0 matters: an associative-array counter that is never incremented

does not exist, so a report built only from existing keys can never show a hole.

7 Design Assertions

Test term — Assertion (SVA) checks. Properties checked on every clock edge during simulation. They report a violated design rule at the cycle it happens — a memory write in REFILL, an acceptance outside IDLE — instead of when a wrong value finally reaches the scoreboard, possibly thousands of cycles later. A cover property shows that a scenario really occurred.

The `mem_if` protocol assertions (Lab V2) already apply on both cache ports. In addition, write at least the following design assertions for the cache controller:

1. No memory-side request in the IDLE state: the memory is touched only during a miss or a flush.
2. Only memory writes in EVICT; only memory reads in REFILL.
3. No CPU request is accepted outside the IDLE state.
4. A CPU request is accepted only on a hit (`cpu.req_ready` implies valid and matching tag for the indexed line).
5. After a refill completes, the line is valid, clean and holds the requested tag.
6. A dirty bit is set only by an accepted CPU write.
7. After `flush_done`, no line is valid or dirty.
8. Bounded latency: a held CPU request is accepted within 2×`LINE_WORDS` plus a small constant cycles (derive the exact bound from your controller and justify it in `doc/verification.md`).

Cover properties: a dirty eviction; a refill with responses in consecutive cycles; a flush that writes back at least two lines.

7.1 Binding from `dv/sva/`

Design assertions on internal state live in a checker module, `hw/ip/dcache/dv/sva/dcache_sva.sv`, attached with a `bind` statement in `dv/sva/dcache_bind.sv`, as in Labs V1 and V2. The RTL is not edited, and the synthesis file list never sees the checker.

```
// hw/ip/dcache/dv/sva/dcache_bind.sv
bind dcache dcache_sva #(
  .ADDR_W(ADDR_W), .LINE_WORDS(LINE_WORDS), .NUM_LINES(NUM_LINES)
) u_dcache_sva (
  .clk, .rst_n, .flush, .flush_done,
  .cpu_req_valid (cpu.req_valid), .cpu_req_ready (cpu.req_ready),
  .cpu_req_we    (cpu.req_we),    .cpu_req_addr  (cpu.req_addr),
  .mem_req_valid (mem.req_valid), .mem_req_ready (mem.req_ready),
  .mem_req_we    (mem.req_we),
  .st_idle  (state_q == IDLE),    .st_evict (state_q == EVICT),
  .st_refill(state_q == REFILL),
  .valid_q, .dirty_q, .tag_q      // internal arrays, names from Lab R4
);
```

```
// in dcache_sva.sv
default clocking cb @(posedge clk); endclocking
default disable iff (!rst_n);

a_mem_idle   : assert property (st_idle |-> !mem_req_valid);
```

```
a_evict_wr   : assert property (st_evict  && mem_req_valid |->  mem_req_we);
a_refill_rd  : assert property (st_refill && mem_req_valid |-> !mem_req_we);
a_accept_idle: assert property (cpu_req_valid && cpu_req_ready |-> st_idle);
a_latency    : assert property (cpu_req_valid && !cpu_req_ready
                                |-> ##[1:MAX_LAT] cpu_req_ready);
c_dirty_evict: cover  property (st_idle ##1 st_evict);
```

Passing state *conditions* (`state_q == IDLE`) instead of the state register keeps the checker independent of the state encoding: adding a state in the RTL does not break the bind. The internal names (`state_q`, `valid_q`, ...) are those of your Lab R4 RTL; document them in `doc/design.md` so that a later change to the RTL knows that a checker depends on them.

7.2 Assertion Fires First

Test term — Negative / error-injection test. A bug is inserted on purpose to prove that the checkers fire: a checker that never fails proves nothing. Record which check or assertion reports the bug first and how long after the faulty cycle; the bug is then removed again.

As in Labs V1 and V2, introduce one controller bug in a local, uncommitted copy (for example, the dirty bit is not cleared after a refill) and record which assertion fires, at which time, and when the scoreboard first reports an error. The difference is usually large for a cache: the assertion fires at the cycle of the bug; the final memory check may only see it at the end of the test.

8 Coverage Closure and the Testplan

8.1 Coverage Closure as the Completion Criterion

Verification of `dcache` is complete when, over the whole regression:

1. every test passes for every seed and every `PARAMS` set;
2. no assertion fails, and every cover property is hit at least once;
3. every bin of the coverage plan (Section 6.2) is hit at least once in *each* `PARAMS` set;
4. every line of the testplan is marked done, with a pointer to the test, assertion or bin that closes it.

Coverage is merged over all runs of one configuration: a bin is closed if any run hits it. With covergroups the simulator merges the databases; with the fallback counters, add a small generic script, `flow/scripts/cov_merge.py`, that sums the `cov_*.txt` files of a regression and prints the holes. The `regress` summary must show the coverage result per configuration.

The closure loop is: run the regression, read the holes, decide for each hole whether the stimulus cannot reach it (change a knob, a constraint or add a directed sequence) or the bin is not reachable by design (remove it from the plan with a written justification). Never close a hole by deleting a bin without that justification. Record which bin was hardest to hit and what you changed.

8.2 The Testplan with Coverage Items

Extend `hw/ip/dcache/dv/testplan.md` in the format introduced in Lab V1. Every feature line now has a third kind of evidence besides tests and assertions: coverage items. Example lines:

Feature	Description	Tests	Assertions	Coverage
Read hit	Data returned one cycle after acceptance	`fits`	`mem_if`	`C1`
Clean miss	Refill of `LINE_WORDS` words, then accept	`sequential`	2, 3, 5	`C1`, `C3`
Dirty miss	Write-back of victim, then refill	`conflict`	2, 6	`C1`, `C4`
Write-back policy	Write hit sets dirty, no memory write	`fits`	1, 6	`C1`
Flush	Write back all dirty lines, invalidate	`flush`	7	`C5`
Back-pressure	Held request accepted in bounded time	all	8	`C4`, `C6`
Perf counters	Equal to the testbench count	all	—	—

Test term — Back-pressure test. The receiving side holds `ready` low and the sender must hold its request stable until it is accepted. Here the cache is the source of back-pressure: on every miss it keeps `cpu.req_ready` low for up to 2×`LINE_WORDS`+2 cycles. Every test exercises it; assertion 8 bounds the wait and the `mem_if` assertions check that the driver holds the request.

The last line shows that a feature may be closed by a check alone: the testbench counts accesses, misses and write-backs itself and compares them with `perf` at the end of every test.

8.3 Regression

Test term — Regression. The whole set of tests × seeds × parameter sets, run by one command (`make regress`) with a pass/fail summary. Here it is also the unit over which coverage is merged: closure is judged on a regression, not on one run.

Extend `dv/regress.list` so that `make regress IP=dcache` runs all six tests × at least 5 seeds × at least three `PARAMS` sets, for example the default, `LINE_WORDS=2 NUM_LINES=32` and `LINE_WORDS=8 NUM_LINES=8`. Long random tests (`+NUM_TXN=20000`) run with fewer seeds; short directed tests with one seed.

9 Measured Performance versus Lab R4 Predictions

In Lab R4 you predicted hit rates by hand for the `sequential` and `conflict` patterns. Now measure them with the performance counters (`perf.accesses`, `perf.misses`, `perf.writebacks`, `perf.stall_cycles`) and with the latency statistics. Before running `fits` and `random`, predict their hit rates too and write the prediction in `doc/verification.md`.

Test	Predicted hit rate (R4)	Measured hit rate	Write-backs	Avg. latency (cycles)	AMAT, formula (cycles)
`sequential`					
`fits`					
`conflict`					
`random`					

Fill the AMAT column with $AMAT = \textit{hit time} + \textit{miss rate} \times \textit{miss penalty}$, using the measured clean and dirty miss penalties weighted by how often each occurred. Then answer:

1. Where does a measured hit rate differ from your prediction, and why? (Typical causes: the first access of each line, the preload, the flush at the end, `wr_pct` changing the dirty fraction.)

2. How do clean and dirty miss penalties depend on `LINE_WORDS`? Measure both for `LINE_WORDS` = 2, 4 and 8 and give a formula.
3. Does the measured average latency match the AMAT formula? If not, which cycles does the formula not count?
4. Do `perf.stall_cycles` and the latency statistics agree? Derive one from the other.

Systematic comparisons of cache configurations (line size versus access pattern, replacement policies, area budget) are done with a model, much faster, in Lab T-AM1; this lab only measures the RTL you built.

10 Stretch Goal: Verifying the 2-Way Cache

If you implemented the 2-way set-associative stretch goal in Lab R4 (`WAYS=2`, one LRU bit per set):

- Extend the tag model with ways and the LRU bit, so the hit/miss check still predicts every access exactly, including which way is evicted.
- Add an assertion that at most one way hits: `$onehot0(way_hit)`, and a cover property for a hit in each way.
- Add `WAYS=2` configurations to the regression and the way to the coverage plan (`C1` crossed with the hit way).
- Rerun `conflict` with two alternating tags and explain the change in hit rate.

11 Requirements / Tasks

1. Create `hw/ip/dcache/dv/` with the structure of Section 12. `make sim IP=dcache` with the default `DV_ENV=dv` runs `smoke`. The Lab R4 smoke testbench in `tb/` must keep running.
2. Write `cache_txn` and `conflict_txn` with the knobs and constraints of this document; check every `randomize()`; use one object per transaction.
3. Implement the frontdoor loader and the preload pattern; optionally the backdoor loader and a run-time comparison.
4. Implement the flat reference model, the tag model and the three checks, including the victim and refill address checks.
5. Implement the CPU and memory monitors, the latency statistics per result class, and the comparison of `perf` with the testbench's own counts at the end of every test.
6. Implement the six tests and the plusargs; print all of them at the start.
7. Implement `dcache_cov` with the full coverage plan, the fallback counters and the `DV_COVERGROUP` covergroup.
8. Write `dcache_sva` with the eight design assertions and three cover properties; bind it from `dv/sva/`. Show one assertion firing on an injected bug.
9. Write `dv/testplan.md` with tests, assertions and coverage items for every feature.
10. Run the regression (three `PARAMS` sets, all tests, at least 5 seeds) to coverage closure; report the holes you closed and how.
11. Fill the performance table and answer the questions of Section 9.
12. Any RTL bug you find is fixed in `hw/ip/dcache/rtl/` in its own commit whose message names the failing test, seed and `PARAMS`; record it in the bug log.

12 Repository and Flow

Files created or changed in this lab:

```
hw/ip/dcache/
+-- Makefile              DV_ENV ?= dv selects TB_FLIST/TB_TOP; include regress.mk
+-- dv/
|   +-- testplan.md       features -> tests / assertions / coverage items
|   +-- regress.list      tests x seeds x PARAMS sets
|   +-- dv.f              -f rtl/dcache.f, -f arb_mem rtl list, tb_utils_pkg, TB files
|   +-- tb/tb_top.sv      clock, reset, mem_if x3, dcache, arb_mem, bind, test start
|   +-- env/
|   |   +-- dcache_txn.svh      cache_txn, conflict_txn
|   |   +-- dcache_drv_mon.svh  driver, loader, CPU and memory monitors (tasks)
|   |   +-- dcache_scb.svh      reference model, tag model, three checks, statistics
|   +-- tests/dcache_tests.svh  smoke, sequential, fits, conflict, random, flush
|   +-- sva/dcache_sva.sv, dcache_bind.sv
|   +-- cov/dcache_cov.svh
+-- doc/verification.md   results, performance table, bug log
+-- rtl/                  changed only to fix bugs (one commit per bug)
flow/scripts/cov_merge.py merges fallback coverage files of a regression
```

The `dv.f` file list includes the RTL file lists with nested `-f`, so `dcache` and `arb_mem` exist exactly once in the repository:

```
// hw/ip/dcache/dv/dv.f
-f ${REPO_ROOT}/hw/ip/arb_mem/rtl/arb_mem.f
-f ${REPO_ROOT}/hw/ip/dcache/rtl/dcache.f
${REPO_ROOT}/dv/common/tb_utils_pkg.sv
+incdir+${REPO_ROOT}/hw/ip/dcache/dv/env
+incdir+${REPO_ROOT}/hw/ip/dcache/dv/tests
+incdir+${REPO_ROOT}/hw/ip/dcache/dv/cov
${REPO_ROOT}/hw/ip/dcache/dv/sva/dcache_sva.sv
${REPO_ROOT}/hw/ip/dcache/dv/sva/dcache_bind.sv
${REPO_ROOT}/hw/ip/dcache/dv/tb/tb_top.sv
```

No new make targets are introduced; this lab uses the targets and variables of Labs L0 and V1. The `dcache` block will have more than one verification environment: its Makefile has an IP-local variable `DV_ENV ?= dv` that selects `TB_FLIST` and `TB_TOP`. This lab provides the default value `dv`; the DV track adds `dv_layered` (Lab T-DV1) and `dv_uvm` (Lab T-DV2) through the same variable. Write the selection so that adding a value is one line:

```
# hw/ip/dcache/Makefile (excerpt)
DV_ENV ?= dv
ifeq ($(DV_ENV),dv)
  TB_FLIST := dv/dv.f
  TB_TOP   := tb_top
endif
```

Example commands:

```
make sim IP=dcache TEST=smoke
make sim IP=dcache TEST=random SEED=7 PLUSARGS="+NUM_TXN=20000 +WR_PCT=30"
make sim IP=dcache TEST=conflict PARAMS="LINE_WORDS=8 NUM_LINES=8" WAVES=1
make sim IP=dcache TEST=random DEFINES=DV_COVERGROUP     # simulators with covergroups
make sim IP=dcache TEST=smoke TB_FLIST=tb/tb.f TB_TOP=tb_dcache   # Lab R4 smoke TB
make regress IP=dcache
$PYTHON flow/scripts/cov_merge.py build/dcache/regress_summary.txt
```

Logs and coverage files land in `build/dcache/<cfg>/sim/<test>_s<seed>.log` and `cov_<test>_s<seed>.txt` in the same directory.

13 Deliverables

The merge request (branch `lab/v3-dcache`, tag `v3-done`) contains:

- The complete `hw/ip/dcache/dv/` tree and the updated `Makefile`; `make lint sim` still passes for every IP (`make check-all`).
- `dv/testplan.md` with every line closed.
- `doc/verification.md` containing:
  - the scoreboard design: why the reference model is a flat memory, and what each of the three checks can and cannot find;
  - the tag model and the observed-result classification;
  - the derivation of the bounded-latency assertion;
  - the regression summary (configurations × tests × seeds) and the merged coverage report showing every planned bin hit, the holes you closed and how;
  - the assertion-fires-first result;
  - the performance table, the predictions and the answers;
  - the bug log: every RTL bug found, how it showed up (check, assertion, test, seed), and the fixing commit.
- `hw/ip/dcache/README.md` updated: how to run the V3 environment, regression and coverage results summary.

14 Assessment

Criterion	**Weight**
Constrained-random classes, tests and plusargs	15%
Scoreboard: three checks, tag model, preload, perf counter check	25%
Functional coverage plan, implementation and closure	20%
Design assertions, bind and cover properties	15%
Testplan and regression across configurations	10%
Measured performance versus predictions	10%
Documentation and bug log	5%
Total	100%

The 2-way stretch goal adds up to 10 bonus points.

15 Review Questions

1. Why can the reference model of a cache be a flat memory? What property of a cache does that rely on?

2. A write is lost because a dirty bit is never set. Which of your three checks finds it, and when?
3. Your cache always misses. Which check finds it?
4. What goes wrong if two transactions in your sequence share one object handle?
5. What does `randomize()` return when the constraints conflict, and what happens if you ignore it?
6. Why must coverage be sampled from observed, checked behavior rather than from the generated stimulus?
7. Which coverage bin was the hardest to hit, and what did you change in the stimulus to hit it?
8. Why is coverage closure, and not a number of transactions, the completion criterion? Can 100% coverage still miss a bug?
9. Why does the bind pass `state_q == IDLE` rather than `state_q`? What would break otherwise?
10. What would break in your testbench if the loader wrote memory while the cache was active? What is this problem called?

16 References

- IEEE Std 1800-2017: Clause 8 (classes), Clause 18 (constrained random value generation), Clause 19 (functional coverage), Clause 23.11 (`bind`).
- Chris Spear and Greg Tumbush, *SystemVerilog for Verification*: chapters on randomization and functional coverage.
- Verification Academy: SystemVerilog classes, constrained randomization, functional coverage and coverage closure material.
- OpenTitan: DV methodology and testplan format (`data/<ip>_testplan.hjson`), https://opentitan.org/book/doc/contributing/dv/methodology/.
- Verilator manual: randomization and coverage support of the installed version.
- Harris and Harris, *Digital Design and Computer Architecture: RISC-V Edition*: chapter on memory systems (AMAT).